DEPLOYMENT PLAN

Coordina AI

Autonomous AI Teammate & Workflow Orchestrator

By Team BU ZHI DAO

UM HACKATHON 2026

Table of Content

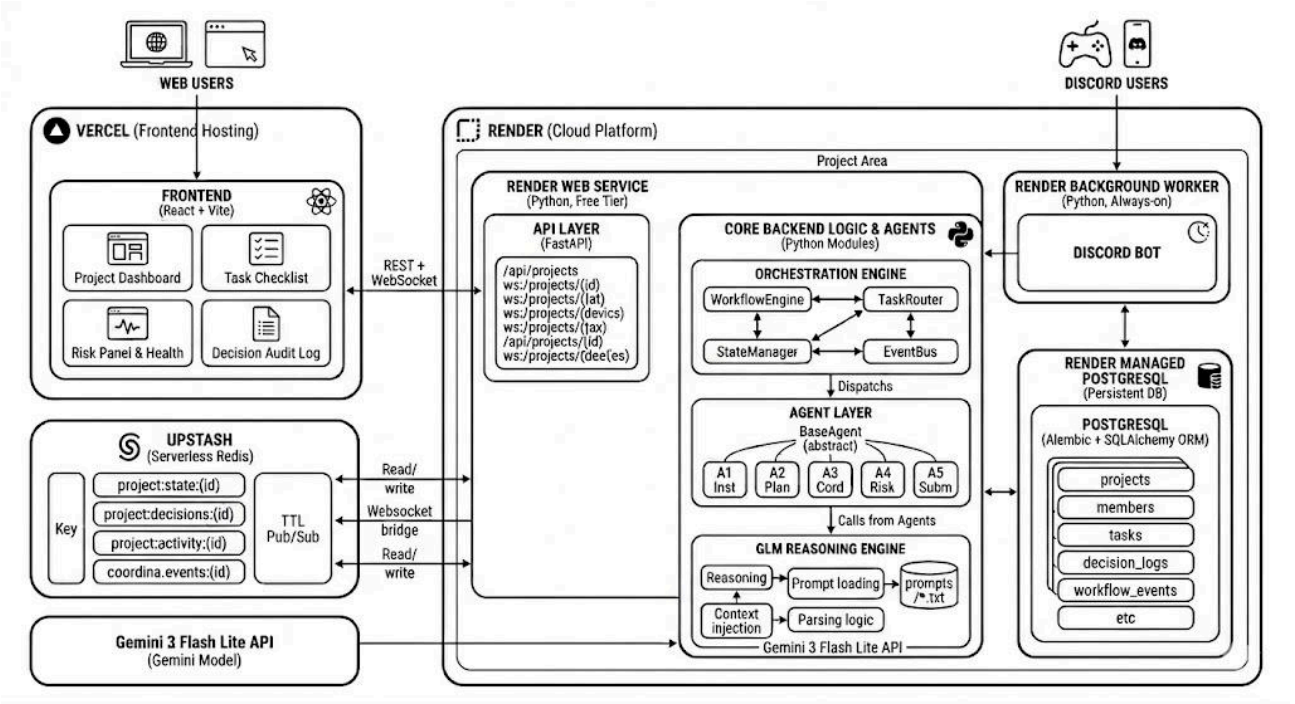
1.0 Purpose and Scope 3
2.0 Production Architecture Overview 4
2.1 System Components 4
2.2 Technology Stack 4
2.3 Scalability Consideration 5
2.3.1 Backend Scaling 5
2.3.2 Database Scaling 5
2.3.3 Frontend Scaling 5
2.3.4 Caching & Rate Limiting 5
3.0 CI/CD Pipeline and Deployment Workflow 6
3.1 Development Environment 6
3.2 Database Migration Preparation 6
3.3 Pull Request Creation 6
3.4 Continuous Integration (Automated Checks) 6
3.5 Code Review & Approval 7
3.6 Database Migration Execution 7
3.7 Production Environment Controls 7
3.8 Post-Deployment Verification 7
4.0 Rollback Plan 7
5.0 Integration Security and Credential Management 8
5.1 Secret Management 8
5.2 Role-Based Access Control (RBAC) 9
5.2.1 GitHub Repository Access 9
5.2.2 Render Access Control 9
5.2.3 Vercel Access Control 10
5.3 Audit Logging and Traceability 10
5.4 Credential Rotation Tracking 10
5.5 Security Monitoring Practices 10
6.0 Production Engineering and Code Maturity 11
6.1 Code Quality Standards 11
6.2 Architecture Maturity 11
6.3 Token Efficiency and Cost Management 12
6.4 Repository Hygiene 12
7.0 Deployment Verification Checklist 14
8.0 Known Risks and Mitigation 16

1.0 Purpose and Scope

Coordina AI is an autonomous AI teammate and workflow orchestration system designed to interpret project briefs and academic rubrics through intelligent instruction analysis, generate structured project plans with automated task decomposition, coordinate team roles and meeting agendas, identify risks with potential deadline failure probabilities, and validate final submissions against rubric requirements to ensure completeness and quality. The deployment scope of this system covers a full-stack architecture consisting of a FastAPI-based asynchronous backend powered by five specialized AI agents, a React + Vite single-page frontend application, a Discord bot operating as a background worker for real-time team communication, a PostgreSQL database managed with schema migrations via Alembic, and a Redis cache layer deployed via Upstash for optimized performance. The system is hosted across multiple platforms, with backend and bot services deployed on Render, the frontend hosted on Vercel, and database services managed through Supabase or Render depending on environment requirements. The deployment lifecycle is structured across three environments: a development environment using local machines and feature branches for iterative testing, an optional staging environment for pre-production validation through separate deployments, and a production environment that serves live users with fully integrated and monitored services.

2.0 Production Architecture Overview

2.1 System Components



2.2 Technology Stack

Component	Technology	Hosting	Rationale
Backend API	FastAPI + Uvicorn	Render Web Service	Async Python framework, free tier support, WebSocket ready
Frontend	React 19 + Vite	Vercel	Optimized React hosting, automatic CI/CD, edge deployment
Database	PostgreSQL 16	Render DB or Supabase	Free managed tier, supports Alembic migrations
Cache	Redis 7	Upstash (Serverless)	Free tier, serverless operation, connection pooling
Discord Bot	Python + discord.py	Render Background Worker	Always-on process using same deployment pipeline
AI Reasoning	Gemini 3 Flash Lite API	External (Google)	Real-time LLM inference accessed securely via API key

2.3 Scalability Consideration

2.3.1 Backend Scaling

- Render Web Service auto-scales based on CPU/memory usage
- Strategy: Monitor response times; upgrade instance if P99 latency > 500ms
- Horizontal scaling: Can run multiple concurrent instances with Render's auto-scaling
- Connection pooling: SQLAlchemy + asyncpg configured for max 20 DB connections per instance
- Redis connection: Upstash supports automatic connection pooling; upgrade tier if 90%+ utilization

2.3.2 Database Scaling

- PostgreSQL (Render/Supabase free tier) supports ~10GB storage
- Plan: Monitor table sizes monthly; prepare migration to paid tier at 70% capacity
- Read replicas: Supabase offers read replicas on paid plans
- Indexing strategy: Ensure all foreign key queries have indexes on project_id, task_id

2.3.3 Frontend Scaling

- Vercel automatically scales edge functions globally
- CDN: Static assets cached at edge; no action required
- Bundle size: Monitor with npm run build output; target < 200KB gzipped

2.3.4 Caching & Rate Limiting

- Redis cache: Workflow state, decision logs, user sessions
- Rate limiting: Implement via Redis at API gateway (e.g., token bucket per user)
- Plan: If > 1000 concurrent users, upgrade Upstash tier or implement local in-memory cache

3.0 CI/CD Pipeline and Deployment Workflow

3.1 Development Environment

- Create a feature branch from main
- Branch Strategy
  - All development occurs in feature/* branches.
  - main remains production-stable.
- Local Development Setup
  - Backend runs on local FastAPI development server.
  - Frontend runs via local Vite dev server.
  - Database: Local Docker PostgreSQL instance.
  - Cache: Local Docker Redis or Upstash free tier.
  - Discord Bot: Optional manual deployment to a development server.
- Testing Responsibilities
  - Write unit tests under tests/unit/.
  - Add integration tests under tests/integration/.
  - Execute tests locally:
  - Run linters and formatting checks (if configured).
  - Commit frequently with descriptive messages.

3.2 Database Migration Preparation

- Generate schema migrations locally
- Apply migration locally
- Validate migration
- Commit migration files

3.3 Pull Request Creation

- Push feature branch
- Open Pull Request targeting main.
- Link relevant issues or tickets.
- Feature enters automated validation pipeline.

3.4 Continuous Integration (Automated Checks)

GitHub Actions automatically performs:

- Full backend and frontend test suite execution.
- Type validation (mypy or Pydantic validation).
- Linting and formatting checks.
- Security scanning (if configured).
- Build verification of frontend and backend artifacts.

Deployment proceeds only if all checks pass.

3.5 Code Review & Approval

- Minimum one approval from technical lead.
- Review focuses on:
  - Code quality
  - Security considerations
  - Architecture consistency
- Feedback addressed via additional commits.
- Force pushes during review are avoided.
- All changes are then merged into the production branch (main), which triggers automatic Continuous Deployment.

3.6 Database Migration Execution

- Build pipeline runs.
- Production migrations executed automatically
- The server starts using an updated schema.
- Deployment rolls back automatically if migration fails.

3.7 Production Environment Controls

- Only the main branch deploys to production.
- All CI checks must pass.
- PR requires at least one technical reviewer approval.
- Render logs monitored for runtime errors.
- Frontend performance tracked via Vercel analytics.
- Error tracking using Sentry.

3.8 Post-Deployment Verification

Immediately after deployment:

- Confirm API documentation accessible at /docs.
- Execute smoke tests using Postman or curl.
- Validate authentication and core workflows.
- Monitor logs and metrics for anomalies.

4.0 Rollback Plan

Scenario	Trigger	Response Time	Procedure

API crashes	5xx errors detected from Render	Immediate (< 5 min)	Revert last commit and redeploy previous stable version
Database migration fails	Alembic migration reports error	Immediate (< 2 min)	Downgrade database schema and investigate migration issue
Frontend broken	White screen or 404 detected on Vercel	Immediate (< 5 min)	Roll back Vercel deployment to last stable build
Memory leak detected	Render instance OOM-killed	< 10 min	Temporarily scale instance resources and deploy memory fix
External API unavailable	Gemini 3 Flash Lite API timeouts	< 15 min	Queue requests and retry using exponential backoff strategy
Data corruption	Integrity errors detected in logs	Immediate + manual	Restore database from backup and perform manual audit

5.0 Integration Security and Credential Management

5.1 Secret Management

- All production secrets are stored in Render / Vercel environment variable dashboards
- Secrets are accessed via environment variables (`$ENV_VAR_NAME`), never hardcoded in source code
- No credentials are committed to the repository at any time
- Mandatory 90-day credential rotation policy for all API keys and sensitive tokens
- GitHub secret scanning enabled to detect accidental leaks in commits
    - If detected: immediately revoke and regenerate affected credentials across all systems

Credential	Purpose	Rotation Policy	Storage Location
GEMINI_API_KEY	Z.AI GLM inference access	Every 90 days or on suspected breach	Render environment variable
DATABASE_URL	PostgreSQL database connection	On role change or security incident	Render env var (Supabase managed)

REDIS_URL	Upstash Redis connection	Every 90 days	Render environment variable
DISCORD_TOKEN	Discord bot authentication	Immediately upon compromise	Discord app (regenerated token)
VERCEL_TOKEN	Vercel deployment API access	Every 90 days	GitHub Actions secret
RENDER_DEPLOY_HOOK	Trigger Render deployments	Every 90 days	GitHub Actions secret
Database backup key	Supabase encryption / backup access	Managed automatically (no manual rotation)	Supabase dashboard

5.2 Role-Based Access Control (RBAC)

5.2.1 GitHub Repository Access

- **Admin (Repo Owner)**
  - Full repository access
  - Can merge to `main`
  - Can rotate secrets and manage security settings
- **Maintainer (Technical Lead)**
  - Can approve pull requests
  - Can trigger deployments
  - Cannot modify organization-level secrets
- **Developer**
  - Can create branches and pull requests
  - Cannot push directly to `main`
  - Cannot deploy production systems
- **CI/CD Bot (GitHub Actions)**
  - Limited to automated build and deployment tasks
  - No access to modify secrets or repository configuration

5.2.2 Render Access Control

- **Owner**: Full system control
- **Maintainer**: Can manage environment variables and restart services
- **Read-Only**: Can view logs only
- **Deployment role**: Can trigger manual deployments without configuration access

5.2.3 Vercel Access Control

- **Owner**: Full access
- **Developer**: Can view deployments and preview builds
- Environment variables are restricted from modification by non-owners

5.3 Audit Logging and Traceability

- All deployments are tracked via **GitHub Actions logs**
- Each deployment records:
  - Deploying actor (e.g., GitHub Actions bot)
  - Timestamp
  - Commit hash
  - Deployment status (success/failure)
- Pull request history provides full code change traceability
- Approval history documents who reviewed and approved changes

5.4 Credential Rotation Tracking

- All sensitive credentials are tracked in a rotation log:
  - Credential name
  - Rotation date
  - Responsible maintainer
  - Reason for rotation (e.g., scheduled / incident response)

5.5 Security Monitoring Practices

- GitHub automatically scans for leaked secrets in commits
- Immediate rotation is enforced if exposure is detected
- Environment variables are isolated per environment (dev / staging / production)

- Production systems are protected by least-privilege access principles

6.0 Production Engineering and Code Maturity

6.1 Code Quality Standards

- Testing Strategy
  - Backend coverage target: ≥70% (100% for critical paths such as agents, API endpoints, orchestrator)
  - Frontend coverage target: ≥50% for components
  - Test layers:
    - Unit tests: isolated function-level validation (tests/unit/)
    - Integration tests: API + database workflows (tests/integration/)
    - End-to-end tests: full user flows (manual/Cypress)
- Coverage enforced via pytest --cov
  - Testing Workflow
  - Fast unit tests run per commit
  - Integration tests run in CI pipeline
  - E2E tests executed before major releases
- Code Style & Formatting
  - Python: black, flake8, isort (PEP8-compliant, line length 100)
  - JavaScript/TypeScript: ESLint with auto-fix enabled
  - Pre-commit enforcement for consistency across contributors
- Documentation Standards
  - Google-style docstrings for all backend functions
  - Mandatory documentation for:
    - API endpoints
    - Environment variables
    - Deployment procedures
- Centralized documentation references (e.g., schema, architecture)

6.2 Architecture Maturity

- Core Design Patterns
  - Dependency Injection (FastAPI Depends)
  - Repository pattern (database abstraction layer)
  - Service layer separation (business logic isolation)
  - Event-driven communication (Redis pub/sub)
  - State machine-based workflow orchestration
  - Fully asynchronous architecture (async/await)
- Modular System Design
  - Clear separation of concerns:
    - api/ → request handling only
    - services/ → business logic
    - models/ → database schema

        - agents/ → AI reasoning modules
        - orchestrator/ → workflow engine
    - No circular dependencies enforced
- Error Handling Strategy
    - Custom exception hierarchy (CoordinaError, GLMError, WorkflowError)
    - Graceful degradation for external API failures
    - Centralized logging for monitoring and debugging

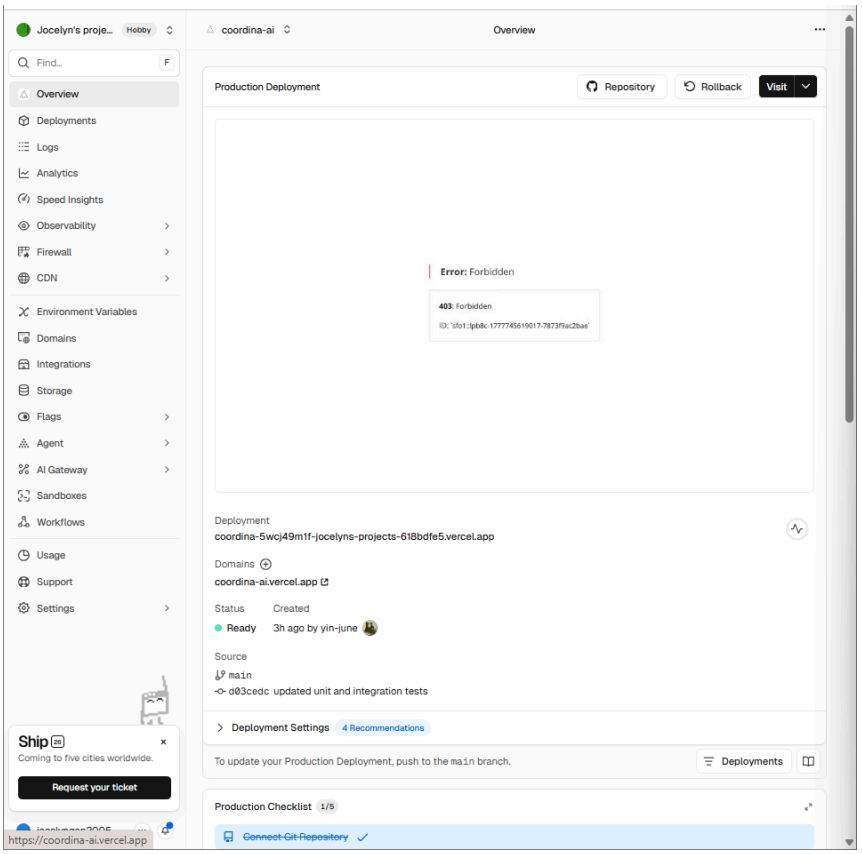
6.3 Token Efficiency and Cost Management

- AI Cost Optimization
    - Redis caching for repeated workflows
    - Hash-based cache keys for duplicate request detection
    - Batch processing for multiple workflows
    - Sampling for analytics instead of full logging
- Infrastructure Cost Control
    - Free-tier-first deployment strategy (Render, Vercel, Upstash)
    - Automatic upgrade triggers based on usage thresholds
    - Regular monitoring of compute, database, and cache usage
    - Estimated baseline cost: ~$50/month at scale
- Performance Optimisation
    - Database indexing for frequent queries
    - SQLAlchemy connection pooling
    - Avoid N+1 queries via eager loading
    - Frontend code splitting and lazy loading (Vite + React Router)
    - Automated bundle size monitoring in CI/CD

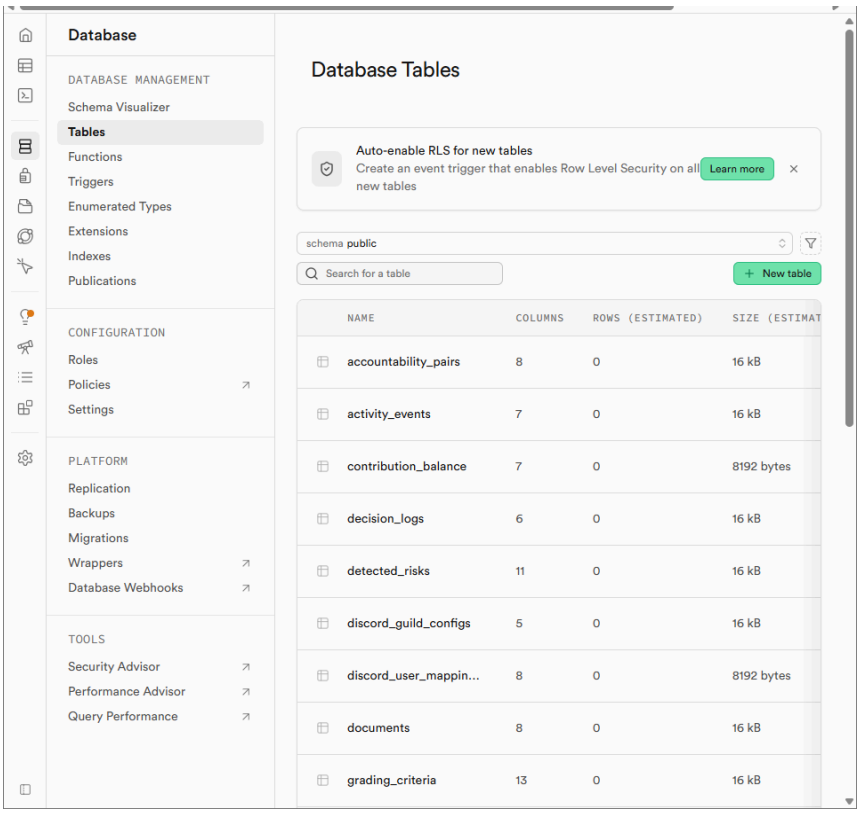
6.4 Repository Hygiene

- Branch Management
    - Feature-based workflow (feature/*)
    - Main branch protected with CI checks and PR reviews
    - Regular cleanup of merged branches
- Commit Standards
    - Squashed or rebased commits before merging
    - No WIP or redundant commit history in main
    - Force-push allowed only on feature branches
- Dependency Management
    - Python: pinned versions in requirements.txt
    - Node: version-controlled package-lock.json
    - Security audits:
        - pip-audit for backend
        - npm audit for frontend
- Regular dependency updates with regression testing

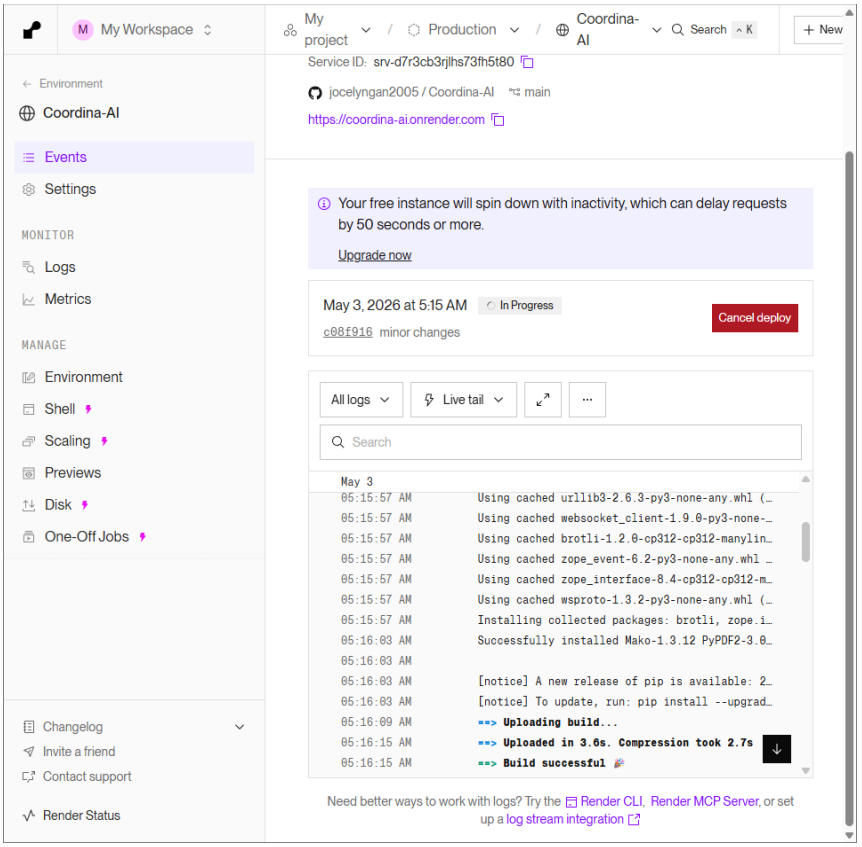
7.0 Deployment Verification Checklist



Frontend builds successfully.



Database migrations completed.



Backend health check passes.

8.0 Known Risks and Mitigation

Risk	Probability	Impact	Mitigation
Gemini API unavailable	Low	Critical	Implement retry logic with exponential backoff, queue requests, and cache previously generated results
Database corruption	Very Low	Critical	Maintain daily backups via Supabase and perform monthly automated point-in-time recovery testing
Redis connection drop	Low	High	Use Redis connection pooling and enable graceful degradation with fallback in-memory caching
Render instance OOM	Medium	High	Monitor memory usage, trigger alerts at 70% utilization, and auto-upgrade instance tier if required
Frontend bundle exceeds 1MB	Low	Medium	Enforce bundle size checks in CI/CD and implement code splitting and lazy loading
CORS misconfiguration	Low	Medium	Whitelist approved frontend origins and validate configuration against deployment URLs
Secrets leaked in logs	Low	Critical	Avoid logging request/response bodies and immediately rotate credentials if exposure occurs
Database connection pool exhausted	Low	Medium	Configure max_overflow=10, monitor active connections, and scale pool size when needed
Cascading system failures	Low	High	Implement frontend error boundaries, offline UI fallback, and client-side request queuing